



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto: Sistema Verificador de Cumplimiento OWASP

Curso: Calidad Y Pruebas De Software

Docente: Mag. Ing. Patrick Cuadros Quiroga

Integrantes:

Concha Llaca, Gerardo Alejandro

(2017057849)

Diego Fabrizio Andia Navarro

(2022073906)

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo	Profesor	-	24/06/2026	Actualización por re-diseño de la persistencia (SQLite3/MySQL), inclusión de la arquitectura de la Extensión de VS Code y el escáner de CVEs

**Sistema Sistema Verificador de Cumplimiento
OWASP
Documento de Arquitectura de Software
Versión 2.0**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo	Profesor	-	24/06/2026	Actualización por re-diseño de la persistencia (SQLite3/MySQL), inclusión de la arquitectura de la Extensión de VS Code y el escáner de CVEs

INDICE GENERAL

Contenido

1. INTRODUCCIÓN	5
1.1. Propósito (Diagrama 4+1)	5
1.2. Alcance	5
1.3. Definición, siglas y abreviaturas.....	5
1.4. Organización del documento	5
2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS	5
2.1.1. Requerimientos Funcionales.....	5
2.1.2. Requerimientos No Funcionales – Atributos de Calidad.....	6
3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA.....	6
3.1. Vista de Caso de uso	6
3.1.1. Diagramas de Casos de uso.....	7
3.2. Vista Lógica	7
3.2.1. Diagrama de Subsistemas (paquetes).....	7
3.2.2. Diagrama de Secuencia (vista de diseño).....	7
3.2.3. Diagrama de Colaboración (vista de diseño).....	7
3.2.4. Diagrama de Objetos	7
3.2.5. Diagrama de Clases	7
3.2.6. Diagrama de Base de datos (relacional o no relacional).....	7
3.3. Vista de Implementación (vista de desarrollo).....	7
3.3.1. Diagrama de arquitectura software (paquetes)	7
3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)	7
3.4. Vista de procesos	8
3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)	8
3.5. Vista de Despliegue (vista física)	8
3.5.1. Diagrama de despliegue	8
4. ATRIBUTOS DE CALIDAD DEL SOFTWARE	8
Escenario de Funcionalidad	8

Escenario de Usabilidad.....8

Escenario de confiabilidad.....9

Escenario de rendimiento.....9

Escenario de mantenibilidad9

Otros Escenarios.....9

1. INTRODUCCIÓN

1.1. Propósito (Diagrama 4+1)

[Se presenta una visión global y resumida de la arquitectura del sistema y de los objetivos generales del diseño. Se describen las influencias con los requisitos funcionales y no funcionales del sistema y las decisiones y prioridades establecidas – eficiencia vs. Portabilidad, por ejemplo.]

1.2. Alcance

[El documento se centrará en el desarrollo de la vista lógica del framework. Se incluyen los aspectos fundamentales del resto de las vistas y se omiten aquellas que no se consideren pertinentes como ser el caso de la vista de procesos.]

1.3. Definición, siglas y abreviaturas

[Este apartado proporciona las definiciones de todos los términos, acrónimos y abreviaturas utilizadas a lo largo del documento y que permiten una interpretación correcta del mismo. Se han de incluir los términos técnicos, caso de uso por ejemplo, y los específicos del entorno del sistema, lector de bandas por ejemplo. Es conveniente ordenarlos alfabéticamente]

1.4. Organización del documento

[Aquí va la organización del proyecto]

2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS

[Establezca las prioridades de los requerimientos y las restricciones del proyecto]

2.1. Priorización de requerimientos

[Se procede a desplegar los requerimientos funcionales y no funcionales desde una perspectiva de priorización, mediante una tabla resumen donde pueda desplegar los requerimientos del sistema de la siguiente forma:]

<i>ID</i>	<i>Descripcion</i>	<i>Prioridad</i>

Asimismo con esta prioridad se definirá el orden de implementación.]

2.1.1. Requerimientos Funcionales

[Definir la prioridad de los requerimientos funcionales.]

<i>ID</i>	<i>Descripcion</i>	<i>Prioridad</i>

2.1.2. Requerimientos No Funcionales – Atributos de Calidad

[Definir la prioridad de los requerimientos NO funcionales.]

ID	Descripcion	Prioridad

[Los Atributos de Calidad (QAs) son propiedades medibles y evaluables de un sistema, estas propiedades son usadas para indicar el grado en que el sistema satisface las necesidades de los stakeholders [Wojcik 2013].

Los QAs además son concebidos como aquellos requerimientos que no son funcionales. De hecho, la funcionalidad es mayormente ortogonal a los QAs; un diseño puede cumplir con la funcionalidad deseada y fallar a la hora de satisfacer sus requerimientos de calidad. De esta manera, se entiende a la funcionalidad como la capacidad del sistema para hacer el trabajo para el cual fue pensado, independientemente de la estructura. Existen QAs mayormente usados que se suelen identificar en numerosos sistemas y se tienen que describir, aunque la lista no es fina ya que muy a menudo hay situaciones en que podrían identificarse y proponerse nuevas propiedades para las diversas necesidades de stakeholders.]

2.2. Restricciones

[Aquí van las restricciones del proyecto]

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

3.1. Vista de Caso de uso

[En esta sección se describen los casos de uso del sistema (nombre de la aplicación), donde se abarcan todas las funcionalidades del sistema, se muestran los actores que interactúan en el sistema y las funcionalidades asociadas; asimismo se listará los casos de uso o escenarios del modelo de casos de uso que representen funcionalidades centrales del sistema final, que requieran una gran cobertura arquitectónica o aquellos que impliquen algún punto especialmente delicado de la arquitectura.

La documentación a incluir en esta sección corresponde a la obtenida como consecuencia de la actividad “Realización de casos de uso”:

- Flujos de eventos- Diseño: descripción textual de cómo se realiza el caso de uso en términos de los objetos que colaboran. Resumen de los diagramas conectados con el caso de uso y explicación de sus relaciones.*
- Diagramas de interacción: Diagramas de secuencia, Diagramas de colaboración, objetos participantes, Diagramas de clases.*
- Requisitos derivados: Descripción textual que recoge todos los requisitos, normalmente los no funcionales, de la realización del caso de uso no que han de tenerse en cuenta durante la implementación]*

3.1.1. Diagramas de Casos de uso

La descripción de la estructura se ilustra utilizando un conjunto de casos de uso escenarios lo que genera una nueva vista. Los escenarios describen secuencia de iteraciones entre objetos y entre procesos. Se utilizan para identificar y validar el diseño de arquitectura.

3.2. Vista Lógica

[La vista lógica se encarga de representar los requerimientos funcionales del sistema. Esta sección describe las partes del diseño del modelo significativas para la arquitectura, tales como subsistemas y paquetes.]

3.2.1. Diagrama de Subsistemas (paquetes)

[Diagrama que define los límites entre el sistema, o parte del sistema, y su ambiente, mostrando las entidades que interactúan con él. Este diagrama es una vista de alto nivel de un sistema.

Asimismo, se debe desplegar las partes arquitectónicamente significativas del modelo de diseño, como ser la descomposición en capas, subsistemas o paquetes. Una vez presentadas estas unidades lógicas principales, se profundiza en ellas hasta el nivel que se considere adecuado.]

3.2.2. Diagrama de Secuencia (vista de diseño)

3.2.3. Diagrama de Colaboración (vista de diseño)

3.2.4. Diagrama de Objetos

3.2.5. Diagrama de Clases

3.2.6. Diagrama de Base de datos (relacional o no relacional)

3.3. Vista de Implementación (vista de desarrollo)

[Se detalla la estructura general del Modelo de Implementación y el mapeo de los subsistemas, paquetes y clases de la Vista Lógica a subsistemas y componentes de implementación de manera más detallada]

3.3.1. Diagrama de arquitectura software (paquetes)

[Se detalla la manera como fue implementado el sistema propuesto, se describe visualmente las capas que tiene el sistema, como están distribuidas y sus principales funciones]

3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)

[Se detalla la manera como fue implementado el sistema propuesto, se describe visualmente las capas que tiene el sistema, como están distribuidas y sus principales funciones]

3.4. Vista de procesos

[Describe la descomposición del sistema procesos pesados. Indica que procesos o grupos de procesos se comunican o interactúan entre sí y los modos en que estos se comunican.]

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)

[Se realizará un diagrama del o los procesos del sistema donde se exponga las actividades donde interviene el sistema propuesto, adicionando diagramas que definan el detalle la descomposición del sistema en procesos pesados. Indica que procesos o grupos de procesos se comunican o interactúan entre sí y los modos en que estos se comunican]

3.5. Vista de Despliegue (vista física)

[Se despliega uno o más escenarios de distribución física del sistema sobre los cuales se ejecutará y hará el despliegue del mismo. Muestra la comunicación entre los diferentes nodos que componen los escenarios antes mencionados, así como el mapeo de los elementos de la Vista de Procesos en dichos nodos]

3.5.1. Diagrama de despliegue

[un diagrama de despliegue, amplía el sistema de software y muestra los contenedores (aplicaciones, almacenamiento de datos, microservicios, etc.) que componen este sistema de software]

4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

[Los Atributos de Calidad (QAs) son propiedades medibles y evaluables de un sistema, estas propiedades son usadas para indicar el grado en que el sistema satisface las necesidades de los stakeholders [Wojcik 2013].

Los QAs además son concebidos como aquellos requerimientos que no son funcionales. De hecho, la funcionalidad es mayormente ortogonal a los QAs; un diseño puede cumplir con la funcionalidad deseada y fallar a la hora de satisfacer sus requerimientos de calidad. De esta manera, se entiende a la funcionalidad como la capacidad del sistema para hacer el trabajo para el cual fue pensado, independientemente de la estructura. Existen QAs mayormente usados que se suelen identificar en numerosos sistemas y se tienen que describir, aunque la lista no es fina ya que muy a menudo hay situaciones en que podrían identificarse y proponerse nuevas propiedades para las diversas necesidades de stakeholders.]

Escenario de Funcionalidad

[se califica de acuerdo con el conjunto de características y capacidades del programa, la generalidad de las funciones que se entregan y la seguridad general del sistema.]

Escenario de Usabilidad

[Este atributo de calidad se refiere a la facilidad con la que un usuario puede aprender a utilizar e interpretar los resultados producidos por un sistema [Barbacci 1995]. Para este atributo de calidad, se suelen considerar diversos aspectos de la interacción humano computadora, tales como: aprendizaje del sistema, utilización eficiente del sistema, minimización del impacto de errores, adaptación del sistema a las necesidades del usuario, confianza y satisfacción, entre otros.]

Escenario de confiabilidad

[Es el equilibrio entre la confidencialidad, la integridad, la irrefutabilidad y la disponibilidad de la información y datos manipulados por el sistema. Se trata del estado de un sistema, el cual puede ser transitorio y volátil. La seguridad de un sistema se caracteriza por mecanismos y técnicas empleados para intentar reducir lo más posible el impacto provocado por un ataque, y las amenazas (entendidas como los caminos mediante los cuales se pueden provocar un ataque). Abarca los planos de observación físico, lógico y humanos. Posee tres tipos de enfoque: prevención, precaución y reacción.]

Escenario de rendimiento

[Se mide con base en la velocidad de procesamiento, el tiempo de respuesta, el uso de recursos, el conjunto y la eficiencia.] (Pressman 2010, pág. 187)

Escenario de mantenibilidad

[Combina la capacidad del programa para ser ampliable (extensibilidad), adaptable y servicial. (Pressman 2010, pág. 187)

Otros Escenarios

[“Otros escenarios como por ejemplo: Performance”

Performance: *El atributo de calidad Performance se refiere a la capacidad de responder, ya sea el tiempo requerido para responder a eventos determinados, o bien, la cantidad de eventos procesados en un intervalo de tiempo dado. La Performance caracteriza la proyección en el tiempo de los servicios entregados por el sistema.]*

1. Introducción

El propósito de este documento es detallar el diseño y la arquitectura global del Ecosistema Verificador de Cumplimiento OWASP. Describe la organización lógica, física, de procesos y de implementación del sistema, facilitando la comprensión del flujo de datos entre la extensión del desarrollador en el IDE y el backend de auditoría.

2. Objetivos y Restricciones Arquitectónicas

El sistema debe operar de forma ágil (<5 segundos por escaneo), portable y sin complejas configuraciones de red, utilizando persistencia local (SQLite3) con soporte opcional a base de datos externa (MySQL).

Priorización de Requerimientos

ID	Descripción del Requerimiento	Prioridad
----	-------------------------------	-----------

RF01	Ingreso y escaneo de código, URLs, repositorios de GitHub.	Alta
RF02	Detección de dependencias vulnerables (CVEs).	Alta
RF03	Seguridad por tokens de API para integración REST.	Alta
RF06	Exportación de reportes PDF interactivos descargables.	Alta
RF07	Subrayado de diagnósticos en tiempo real en editor VS Code.	Alta
RNF01	Tiempo de respuesta de análisis de código menor a 5 segundos.	Media
RNF04	Portabilidad multiplataforma mediante SQLite3 local.	Alta

3. Representación de la Arquitectura del Sistema

3.0 Patrón Arquitectónico: Arquitectura en Capas (Layered)

El sistema de software OWASP Verificator se estructura en cuatro capas lógicas:

1. Capa de Presentación (Vista): Jinja2 Templates, Hojas de estilo CSS y Extensión VS Code (Webview).
2. Capa de Controladores (Ruteo / API REST): Routers en app/routers/ (analysis.py, dashboard.py, reports.py).
3. Capa de Lógica de Negocio (Servicios): Módulos en app/services/ (scanner.py, cve_analyzer.py, pdf_export.py).
4. Capa de Acceso a Datos / Persistencia: Módulo app/store.py (InMemoryScanStore) y base de datos SQLite/MySQL.

3.1 Vista de Casos de Uso (Contexto C4 - Nivel 1)

[DIAGRAMA: diagrama_contexto.png]

Representa las fronteras del sistema, identificando los actores (Usuario Desarrollador y Administrador) y los sistemas externos integrados (API de GitHub).

3.2 Vista de Contenedores (C4 - Nivel 2)

[DIAGRAMA: diagrama_contenedores.png]

Representa la distribución a nivel de ejecutables independientes (Dashboard Web, Extensión VS Code, CLI, API FastAPI, MySQL y SQLite).

3.3 Vista de Componentes (C4 - Nivel 3)

[DIAGRAMA: diagrama_de_componentes.png]

Representa a bajo nivel de diseño todos los archivos de código del backend y sus interfaces de puertos (sockets y lollipops).

3.4 Vista Lógica (Diagrama de Clases)

[DIAGRAMA: diagrama_clases.png]

Representa 100% de la jerarquía de clases reales de Python que implementan el negocio (Finding, Scan, APIToken, InMemoryScanStore, PDFReportGenerator).

3.5 Vista de Implementación (Diagrama de Paquetes)

[DIAGRAMA: diagrama_de_paquetes.png]

Representa la estructura física en disco del repositorio completo (raíz, app, data, tests, vscode-extension).

3.6 Vista de Despliegue (Diagrama de Despliegue Físico)

[DIAGRAMA: diagrama_despliegue.png]

Mapea la distribución física de red entre el PC del desarrollador local (VS Code/Browser) y la Máquina Virtual Debian en la nube (FastAPI/MySQL).

4. Atributos de Calidad del Software

1. Funcionalidad: Garantiza el escaneo correcto de OWASP y dependencias CVE mediante motor estático regex.
2. Usabilidad: Feedback instantáneo en tiempo real (diagnósticos y subrayados) en el IDE del programador.
3. Confiabilidad: Hashing de contraseñas, validación robusta contra SSRF en URL, control estricto de expiración de sesiones.
4. Rendimiento: Análisis y carga rápida de endpoints REST bajo Uvicorn/FastAPI.
5. Mantenibilidad: Aislamiento completo de la persistencia mediante InMemoryScanStore en store.py, permitiendo intercambiar el gestor sin alterar los servicios.